

# Compiler Design Lab

## A 12-Week Plan Built Around TinyCStr (SLY)

N.Venkatesh

Compiler Design Laboratory

July 19, 2026

### Course Overview

The goal of this course is to build a real, working compiler for **TinyCStr**, a small statically-typed procedural language( for more details : <https://tinyurl.com/TinyCStr>). The course is inspired by the structure of Prof. Uday Khedker's *Implementation of Programming Languages (theory and lab)* at IIT Bombay (<https://www.cse.iitb.ac.in/~uday/sclp-web/#About>), adapted to use **SLY (Sly Lex Yacc)** (<https://sly.readthedocs.io/en/latest/sly.html>) instead of flex/bison, so that the whole pipeline — lexer, parser, AST, three-address code, and MIPS assembly code generation — is implemented in pure Python.

### Pedagogical strategy: incremental language levels

Rather than Considering the full TinyCStr grammar in one shot, the language is unlocked in four increasing levels. Each level is carried end-to-end (lexer → parser/AST → 3-address code → MIPS) before the next level's constructs are added, so students always own a complete, working compiler rather than a half-finished one.

| Level | New language constructs                                                                                                                                              | Compiler capability added                                            |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| L1    | <code>int</code> only; declarations; assignment; arithmetic (+ - * /); <code>print</code> ; single <code>main</code> function; single-line comments                  | Full pipeline for straight-line integer code                         |
| L2    | <code>double</code> , <code>char</code> , <code>string</code> ; relational ops; ternary <code>?:</code> ; type casting ( <code>double</code> ), ( <code>int</code> ) | Type checking, mixed-type 3AC, multiple MIPS data sections           |
| L3    | <code>if-else</code> , <code>while</code> ; global declarations                                                                                                      | Control-flow graphs, labels/branches in 3AC and MIPS                 |
| L4    | Multiple functions, parameters, <code>return</code> , function calls                                                                                                 | Symbol table scoping, activation records, calling convention on MIPS |

### Within-level staging: simplest construct first

Each level itself bundles more than one new idea (e.g. L1 combines declarations *and* operator precedence; L2 combines four types, type casting, and the ternary operator). To avoid stacking two hard new concepts into the same lab session, every level is internally staged from a trivial sub-case up to the full level, so exactly one hard concept is introduced at a time. The weekly plan below builds each sub-stage in order within its level's weeks.

| Lvl | Stage | Scope                                                                                                | New concept isolated                                                                             |
|-----|-------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| L1  | 1a    | Declarations, assignment of a <i>literal only</i> ( <code>a = 5;</code> ), <code>print</code>        | SLY mechanics end-to-end, with zero grammar ambiguity                                            |
|     | 1b    | Add <code>+</code> <code>-</code> <code>*</code> <code>/</code> , parentheses, recursive expressions | Operator precedence / associativity, precedence-climbing grammars                                |
| L2  | 2a    | Add <code>double</code> only, reusing L1b's expression grammar                                       | Second type declared, no mixed-type arithmetic yet                                               |
|     | 2b    | Add <code>char</code> , <code>string</code> , relational operators                                   | New literal forms and comparison, still no mixed-type arithmetic                                 |
|     | 2c    | Add casting <code>(double)/(int)</code> and ternary <code>?:</code>                                  | Type promotion rules and a value-producing conditional expression                                |
| L3  | 3a    | Plain <code>if</code> (no <code>else</code> ) over L1/L2 statements                                  | Single-branch control flow, one new label pattern                                                |
|     | 3b    | Add <code>else</code> and <code>while</code>                                                         | Dangling-else resolution and loop-back branching, once single-branch <code>if</code> is solid    |
| L4  | 4a    | One extra function, no parameters, no return value                                                   | Multi-function symbol table scoping and a bare <code>jal/jr</code> call, no argument passing yet |
|     | 4b    | Add parameters and <code>return</code>                                                               | Full MIPS calling convention ( <code>\$a0--\$a3</code> , <code>\$v0</code> , stack frame)        |

### Weekly rhythm

Every week follows the same rhythm: a 3-hour supervised lab session building/extending one compiler stage, followed by a take-home extension exercise due at the start of next week's lab (checked in the first 30 minutes). Weeks 1–11 build the compiler incrementally; Week 12 is a dedicated code-optimization module layered on top of the finished L1–L4 compiler, closing with an integrated demo.

## 1 Week-by-Week Plan

### Week 1: Project Setup & TinyCStr Orientation

#### In-lab (3 hrs):

1. Environment setup: Install Python 3.6+, `pip install sly` to use Sly Lex Yacc(SLY), install `spim` (or `QtSpim`) to run MIPS programs
2. Read Slides 1–3 of the TinyCStr specification (procedural, statically typed, its operator set, its four statement kinds); walk through Examples 1–4
3. Understand The overall pipeline to build real working compiler: lexer → parser → semantic analysis → 3-address code → optimizer → MIPS code generator
4. Demo SLY arithmetic expression calculator to understand the working of Lexer/Parser pair of SLY.

**Take-home (due next week):** Re-read the TinyCStr specification and write observations, write 2–3 example TinyStr programs for each level 1–4. **Deliverable:** TinyCStr spec read-through notes, Full working arithmetic expression calculator

### Week 2: Level 1 Lexer (Stages 1a → 1b)

#### In-lab (3 hrs):

1. Issues need to be considered in implementing Lexer for Level 1 using SLY's `Lexer` class: tokens, regex rules via decorators/attributes, keyword vs. identifier disambiguation, ignoring whitespace, `//` comments, and line tracking for error messages.
2. **Stage 1a first:** define only the token set needed for simple assignment statements: `INT`, `ID`, `NUMBER`, `PRINT`, `ASSIGN`, `SEMI`, `LBRACE/RBRACE`, `COMMA`. Test it standalone on a trivial program (`int a; a = 5; print a;`) and confirm the full token stream is correct.
3. **Then Stage 1b:** add `PLUS/MINUS/TIMES/DIVIDE` and `LPAREN/RPAREN`, and re-tokenize Example 1 (Slide 4), which uses arithmetic.
4. Add lexical error handling (illegal character reporting with line number).

**Take-home (due next week):** Write 5 additional test programs spanning both stages, including at least one with a deliberate illegal character, and show the lexer's error output. **Deliverable:** `tinycstr_lexer.py` covering Stages 1a and 1b + a token-dump test report for 6 programs.

### Week 3: Level 1 Parser and AST (Stages 1a → 1b)

#### In-lab (3 hrs):

1. Issues need be considered in implementing Parser for Level 1 using SLY's `Parser` class: grammar rules as decorated methods, building an explicit AST (Python classes / `dataclasses` for `Assign`, `Print`, `Num`, `Var`) instead of just returning values, and data structures for representing program, function, and symbol table.
2. **Stage 1a first:** write the grammar for one `main` function containing a sequence of declaration / simple-assignment / print statements only. Get this parsing and producing an AST for the Stage-1a test program before moving on.
3. **Then Stage 1b:** extend `expr` to a recursive rule over `+` `-` `*` `/` and parentheses; introduce SLY precedence declarations to resolve the resulting shift-reduce conflicts, and inspect SLY's parser debug log to see the conflicts being resolved.
4. Pretty-print the AST (indented text or Graphviz DOT) for Example 1, now that

Stage 1b is in place.

**Take-home (due next week):** Parse Example 2's expression `c=a+b*a+a*a`; and produce the AST; verify operator precedence and associativity are correct by hand-tracing evaluation order. Run on five additional test programs, Add a `--dot` flag that emits a Graphviz picture of the AST. **Deliverable:** `tinycstr_parser.py` covering Stages 1a and 1b, producing AST + rendered AST diagram for Examples 1 and 2.

## Week 4: Level 1: Three-Address Code and MIPS/SPIM

### In-lab (3 hrs):

1. Define a Three-address code IR: `t1 = a + b`, `a = t1`, temporaries, and a `PRINT` instruction. Write an AST-walking code generator that emits Three-address code for assignments, and print.
2. Introduce MIPS essentials needed: registers (`$t0--$t9`, `$v0`), `.data/.text` sections, `li`, `move`, `add/sub/mul/div`, `syscall` codes for integer print (`$v0=1`) and exit (`$v0=10`).
3. Write a simple `3ac_to_mips.py` that maps each Three-address instruction to one or a few MIPS instructions, allocating one memory word per declared variable.
4. Run the generated `.s` file on SPIM/QtSpim for Examples 1 and 2; verify printed output matches hand-computed values.

**Take-home (due next week):** Run and test on five additional examples **Deliverable:** End-to-end L1 compiler: TinyCStr(L1) source → Three-address → MIPS → verified SPIM run, for at least 5 programs.

## Week 5: Level 2 Lexer + Parser: Types, Casting, Relational/Ternary Operators (Stages 2a → 2c)

### In-lab (3 hrs):

1. **Stage 2a first:** add just the `DOUBLE` keyword and floating-point literals to the lexer, and `double` declarations to the grammar/AST, reusing the L1b expression grammar unchanged. Write and parse a program that declares and prints `int` and `double` variables *separately* (no expression mixes the two types yet) — the only new idea this stage is a second declared type, not mixed-type arithmetic.
2. **Stage 2b:** add `CHAR`, `STRING` keywords/literals (`'x'`, `"..."`) and relational operators (`<` `>` `<=` `>=` `==` `!=`); extend the AST with a `RelOp` node. Still no type mixing inside one expression.
3. **Stage 2c:** add cast syntax (`double`), (`int`) and the ternary operator (`?, :`); extend the AST with `Cast` and `Ternary` nodes — this is where mixed-type expressions first become legal, so it is deliberately the last sub-stage..

**Take-home (due next week):** Write 5 TinyCStr(L2) programs, one per sub-stage plus one combining all of them(exercising every new operator and every type, including at least one nested ternary and one chained cast). Produce their ASTs. **Deliverable:** `tinycstr_lexer.py` / `tinycstr_parser.py` upgraded through Stages 2a–2c, tested on Slide 6's Example 3 plus 5 new test cases.

## Week 6: Level 2: Semantic Analysis and Typed Three-Address Code

### In-lab (3 hrs):

1. Use the symbol table (name  $\rightarrow$  type) information; implement type-checking rules: arithmetic between `int`/`double` requires an explicit or implicit promotion rule (assume TinyCStr's promotion policy is similar to C language), relational ops require comparable operands, ternary branches must unify to one type. modify AST if required.
2. Report undeclared-variable and type-mismatch errors with line numbers.

**Take-home (due next week):** Run the type checker over all Week 5 test programs and fix any it flags. **Deliverable:** Symbol-table-based semantic checker for L2, with a written report of at least 3 caught type errors.

## Week 7: Level 2: MIPS Code Generation for Mixed Types

### In-lab (3 hrs):

1. Extend the MIPS back end: floating-point registers (`$f0--$f31`), `l.d/s.d`, `add.d/sub.d/mul.d/div.d`, integer $\leftrightarrow$ double conversion instructions (`cvt.d.w`, `cvt.w.d`), and `syscall` codes for printing a double (`$v0=3`) and a string (`$v0=4`, requiring a `.asciiz` entry in `.data`).
2. Map typed 3AC (from Week 6) to the correct instruction family based on temporary type; implement ternary as a branch-based MIPS sequence.
3. Run Example 3 (Slide 6) end-to-end on SPIM; verify the branch taken and the printed double match the hand-computed value.

**Take-home (due next week):** Compile and verify all 4 Week-5 test programs end-to-end on SPIM. Write a short note on one MIPS floating-point pitfall encountered (e.g. register class mismatch, missing `.data` entry) and how it was fixed. **Deliverable:** Complete L1+L2 compiler pipeline, SPIM-verified on  $\geq 6$  programs total.

**Toolchain Survey** A tour of `gcc`, `g++`, `clang`, `clang++`, `javac`, and `python` to understand the stages in a production compiler. This as a self-contained  $\sim 1$  hour add-on to Week 7's lab and it does not carry a graded weekly deliverable of its own.

1. Compile and run the same small program (arithmetic + a function call) through `gcc`, `g++`, `clang`, `clang++` with varying flags (`-E` for preprocessor output, `-S` for assembly, `-O0` vs `-O2`, `-v` for verbose linker/compile stages). Compare with `javac` + `javap -c` and `python -m dis` to see the analogous stages (preprocessing, front end, IR, optimization, back end) across ecosystems.
2. For each toolchain, map the observed artifact back to the pipeline stage that you have already implemented: `-E` output  $\leftrightarrow$  their lexer's input after comment stripping; `-S` at `-O0`  $\leftrightarrow$  their unoptimized MIPS from Week 7; `-S` at `-O2`  $\leftrightarrow$  a preview of what Week 12's optimization passes will do.
3. Understand the concrete difference between `-O0` and `-O2` assembly for the same source line (e.g. a folded constant or an eliminated dead store) as a motivating teaser for Week 12.

*Optional take-home:* write up the flag  $\rightarrow$  stage  $\rightarrow$  artifact comparison as a short table, for extra credit.

**Week 8: Level 3 Lexer + Parser: Control Flow (Stages 3a → 3b)****In-lab (3 hrs):**

1. **Stage 3a first:** extend lexer/grammar for plain `if ( expr ) stmt` (*no else* yet) and block statements `{ ... }`; extend the AST with `If` and `Block` nodes. Parse a simple single-branch program (e.g. set a flag variable if `a > b`) — this isolates the one new grammar pattern (statement guarded by a condition) with no ambiguity to resolve.
2. **Stage 3b:** add `else` and `while ( expr ) stmt`, plus global declarations (already appearing in Examples 3) formalized as a grammar rule preceding function definitions. Handle the dangling-else ambiguity explicitly (resolve via SLY precedence trick or a %prec-style rule).
3. Parse Example 3 (Slide 6, the `if/else` cast example) and a test example programs covering `while`-loop program (e.g. `sum 1..N`) through to AST.

**Take-home (due next week):** Write a TinyCStr(L3) program with nested `if` inside a `while` loop (e.g. count evens vs odds from `1..N`) and produce its AST diagram.

**Deliverable:** Lexer/parser upgraded through Stages 3a–3b; dangling-else write-up; AST for 2 control-flow programs.

**Week 9: Level 3: Control-Flow Three-address and MIPS Branching****In-lab (3 hrs):**

1. Extend Three-address code with labels and jumps: `if_false t1 goto L1, goto L2`, label placement for `if/else` and `while`; generate a fresh-label allocator.
2. Draw the control-flow graph (CFG) for the Week 8 `while` program by hand before coding, to connect the textbook CFG concept to the generated Three-address code.
3. Map Three-address code labels/jumps to MIPS labels and branch instructions (`beq/bne/blt/bge, j`); regenerate and run the Week 8 programs on SPIM.

**Take-home (due next week):** Extend the compiler to correctly handle a program with an `if` nested inside a `while` nested inside an `if` (3 levels), and confirm the SPIM output matches a hand trace. **Deliverable:** Complete L1+L2+L3 compiler; CFG diagrams; SPIM-verified control-flow programs.

**Week 10: Level 4 Lexer + Parser: Functions (Stages 4a → 4b)****In-lab (3 hrs):**

1. **Stage 4a first:** extend the grammar for a second function with *no* parameters and no return value (a plain helper called from `main`, e.g. `printSquareOfFive()`; require exactly one `main` and detect its absence as an error. Extend the AST with `FunctionCall` nodes, and extend the symbol table to a local scope.
2. **Stage 4b:** add typed parameter lists (function declarations/prototypes as in Slide 7: `int add(int,int);`) and `return` statements. Extend `FuncDecl` with `params/returnType` and add a `Return` node.
3. Parse Example 4 (Slide 7) end-to-end to AST once Stage 4b is in place.

**Take-home (due next week):** Write two more multi-function TinyCStr(L4) programs (e.g. a recursive-looking but loop-based factorial helper, and a two-function min/max program) and produce their ASTs and symbol tables. **Deliverable:** Lexer/parser/semantic-checker upgraded through Stages 4a–4b; documented list of



bugs found and fixed in Example 4.

## Week 11: Level 4: Activation Records and Full MIPS Calling Convention

### In-lab (3 hrs):

1. Introduce activation records: stack frame layout, `$sp/$fp`, MIPS calling convention (`$a0--$a3` for the first four arguments, `$v0` for the integer return value, `$ra` via `jal/jr`, saving/restoring callee-saved registers).
2. Extend three-address code with `PARAM`, `CALL`, and `RETURN` instructions; extend the MIPS back end to emit prologue/epilogue code and `jal/jr $ra` sequences.
3. Compile and run Example 4 (Slide 7, corrected) plus the Week 10 take-home programs on SPIM; verify argument passing and return values.

**Take-home (due next week):** Extend the compiler to support a function with more local variables than fit comfortably in registers, forcing at least one stack-resident local, and confirm correct behavior on SPIM. **Deliverable:** Complete L1–L4 TinyCStr compiler: full pipeline from source to SPIM-verified MIPS for functions with parameters and return values.

## Week 12: Code Optimization Techniques + Integrated Demo

### In-lab (3 hrs):

1. **Build the CFG from three-address code.** Implement leader identification (first instruction; any jump/branch target; any instruction immediately following a branch), partition the three-address code into basic blocks, and construct block-level predecessor/successor edges (fall-through edges and explicit jump/branch edges). Test the builder on the Week 9 nested `if-in-while` program and compare the generated CFG to that week's hand-drawn diagram.
2. **Local optimizations within a basic block.** With the CFG in place, implement **constant folding** (evaluate constant subexpressions at compile time) and **copy propagation** (replace uses of `x` after `x = y` with `y` where safe), scoped to a single basic block first.
3. **Common subexpression elimination via available expressions.** Implement the classic forward, must-analysis data-flow equations (*gen/kill* per block, iterate to a fixed point over the CFG) to compute which expressions are available on entry to each block; reuse a temporary already holding an identical, still-available computation instead of recomputing it. Explicitly discuss why a single-block version of this pass would miss (or wrongly reuse) subexpressions that cross a branch.
4. **Dead code elimination via liveness analysis.** Implement backward, may-analysis liveness data-flow equations over the CFG (a variable is live at a point if some path from that point uses it before redefining it); remove assignments to variables that are dead immediately afterward on *every* outgoing path.

**Deliverable:** CFG builder + all four optimization passes, each with a before/after three-address code (and CFG, where applicable) trace and a generated-MIPS instruction-count comparison; live demo; final compiler repository submission.